

My Week 3 Collaborative Filtering Movie Recommender

Student report by Petr Vorozhtsov | 30 September 2026

My project: github.com/PetrVorozhtsov/RecSys-LLMs | My code revision: b14047a

1. What I completed

I reused MovieLens 100K from Week 2 and built both User-Based and Item-Based collaborative filtering. I kept the two Top-5 lists side by side, added evidence counts, and evaluated predictions against ratings hidden from the model.

Users	Movies	Ratings	Observed matrix
943	1,682	100,000	6.30%

2. My data and sparse-rating strategy

I verified that **u.data** and **u.item** are byte-identical to Week 2. I store missing values as zero because real ratings are 1-5. I chose the assignment's common-rating weighting strategy for both user and item similarities:

similarity = cosine over shared ratings x n / (n + 10), where n is the number of common ratings. I require $n \geq 2$; a single shared rating no longer creates perfect similarity. I use the training-set mean as a small prediction prior and require at least two contributing matches. Item candidates also need five community ratings.

3. What I changed in my code

File	My change and purpose
data.js	I validated rating rows, preserved movie title encoding, and counted training ratings.
script.js	I added overlap weighting, cautious predictions, evidence counts, and reusable item-similarity caching.
index.html / style.css	I clarified the predicted-score scale and displayed support without hiding long titles.
evaluate.js / test	I added a deterministic hidden-rating comparison, timings, and five behavior checks.

4. My experiment - User 1

I selected User 1, who had 272 ratings in MovieLens. The capture below comes directly from my running local page after I clicked Get Recommendations.

Choose an active user

User ID

User 1

Get Recommendations

Recommendations generated for User 1.

User-Based CF

Overlap-weighted ratings from the Top-20 nearest users.

Because this user is similar to other users, we recommend:

Braindead (1992)	4.603
3 supporting matches	
Paradise Lost: The Child Murders at Robin Hood Hills (1996)	4.602
3 supporting matches	
Casablanca (1942)	4.555
16 supporting matches	
Rear Window (1954)	4.495
16 supporting matches	
Hamlet (1996)	4.492
8 supporting matches	

Item-Based CF

Overlap-weighted matches to rated movies; candidates need at least five community ratings.

Because this user liked similar movies, we recommend:

Magic Hour, The (1998)	4.184
107 supporting matches	
Hearts and Minds (1996)	4.143
94 supporting matches	
A Chef in Love (1996)	4.112
82 supporting matches	
Faust (1994)	4.092
88 supporting matches	
Aparajito (1956)	4.085
119 supporting matches	

Figure 1. My live User 1 result: two Top-5 lists, predicted scores, and support counts.

My first User-Based result was **Braindead (4.603; 3 matches)**. My first Item-Based result was **Magic Hour (4.184; 107 matches)**. The different lists show that user and item evidence do not rank movies identically.

I read these numbers as model predictions with different amounts of support, not as proof that one of the recommended films will be liked. I use hidden real ratings for the accuracy comparison on page 4.

5. My experiment - User 196

I selected User 196, who had 39 ratings in MovieLens. The capture below comes directly from my running local page after I clicked Get Recommendations.

Choose an active user

User ID

User 196

Get Recommendations

Recommendations generated for User 196.

User-Based CF

Overlap-weighted ratings from the Top-20 nearest users.

Because this user is similar to other users, we recommend:

All About Eve (1950)	4.661
8 supporting matches	
Godfather, The (1972)	4.648
16 supporting matches	
Annie Hall (1977)	4.589
11 supporting matches	
Godfather: Part II, The (1974)	4.580
13 supporting matches	
Mr. Smith Goes to Washington (1939)	4.575
6 supporting matches	

Item-Based CF

Overlap-weighted matches to rated movies; candidates need at least five community ratings.

Because this user liked similar movies, we recommend:

Dingo (1992)	3.969
14 supporting matches	
Stonewall (1995)	3.961
18 supporting matches	
Gumby: The Movie (1995)	3.936
14 supporting matches	
Wonderful, Horrible Life of Leni Riefenstahl, The (1993)	3.925
25 supporting matches	
Kim (1950)	3.891
23 supporting matches	

Figure 2. My live User 196 result: two Top-5 lists, predicted scores, and support counts.

My first User-Based result was **All About Eve (4.661; 8 matches)**. My first Item-Based result was **Dingo (3.969; 14 matches)**. The smaller profile still gives full lists, while the evidence counts show how much supports each score.

I read these numbers as model predictions with different amounts of support, not as proof that one of the recommended films will be liked. I use hidden real ratings for the accuracy comparison on page 4.

6. My measured comparison

I hid five actual ratings from each of 50 deterministically selected users (250 ratings total). I rebuilt the matrix, item counts, prior, and caches from the remaining 99,750 ratings. The target movie originally had at least ten ratings, so this checks existing users and items. Lower MAE/RMSE is better; coverage is the share for which I could predict.

Method	Predicted / 250	Coverage	Paired MAE	Paired RMSE
User-Based	248	99.2%	0.838	1.050
Item-Based	250	100.0%	0.790	0.962

The paired error columns use the same 248 hidden ratings for both methods. In this sample, Item-Based had lower error. This tests predicted ratings, not whether unobserved Top-5 movies would be liked.

7. My efficiency check

Full-data Top-5 method	Median time across Users 1, 100, 196, 300, 500
User-Based	12.6 ms
Item-Based, cold cache	351.3 ms
Item-Based, warm cache	56.3 ms

I measured on Windows with Node 24.19.0. The cold item method was slower here; reusing cached similarities made it faster on a repeated request. Full user-pair work scales roughly with U squared times I , while full item-pair work scales with I squared times U . Here $I=1,682$ exceeds $U=943$; when users greatly outnumber items, the item approach can be cheaper. These measured times are machine-dependent.

8. Trade-offs, limitations, and reproducibility

I chose **common-rating weighting** because it lowers trust in tiny overlaps without inventing missing ratings. Mean imputation is easy but can pull tastes toward averages. Matrix factorization can capture latent patterns but needs training and tuning. A new user or movie has no collaborative evidence, and this 250-rating warm-start check does not measure cold-start or full Top-5 ranking quality.

To reproduce my work, I run `cd week3, python -m http.server 8000`, then open `http://localhost:8000/`. I run `node evaluate.js` and `node --test recommender.test.js` for the results and checks. The saved metrics are in `evaluation-results.json`; the full browser captures are in `report_assets/`.

9. My conclusion

I completed both CF approaches, tested their predictions on withheld data, and made small-overlap evidence explicit. On this sample Item-Based had lower prediction error, while User-Based was faster on this dataset. I would test ranking relevance and true cold-start users before making a production quality claim.